

Методическое указание к Практической работе №11

по дисциплине «Разработка кроссплатформенных мобильных приложений»

План практической работы:

- Принцип определения сложности архитектуры приложения
- Базовое распределение архитектуры на слои
- Применение архитектуры Clean Architecture
- Реализация слоя Models
- Применение архитектуры Repository
- Выполнение практической работы №11;

Последовательность выполнения практической работы:

1. Принцип определения сложности архитектуры приложения

Первый и фундаментальный шаг в проектировании архитектуры — это адекватная оценка сложности приложения, над которым ведется работа. Архитектура не должна быть избыточной для простого проекта, но и не может быть примитивной для сложной системы. Ключевой ошибкой начинающих разработчиков является либо игнорирование архитектурных принципов на ранних этапах, что приводит к хаотическому росту кодовой базы, либо наоборот — преждевременное внедрение сложных паттернов, которое создает излишнюю нагрузку на процесс разработки.

Сложность приложения оценивается по совокупности нескольких взаимосвязанных факторов. Наиболее очевидным из них является функциональный масштаб, который можно грубо измерить количеством экранов и уникальных бизнес-сценариев. Приложение с одним-двумя экранами, выполняющее простую задачу (например, отображение текущего курса валют), имеет низкую сложность. Приложение с десятком экранов, несколькими типами пользователей и сложными workflows (например, маркетплейс с корзиной, историей заказов, системой отзывов и чатом поддержки) — это проект высокой сложности.

Второй критически важный фактор — стабильность требований и предполагаемый жизненный цикл. Если приложение разрабатывается как прототип или учебный проект с коротким сроком жизни, инвестиции в сложную архитектуру могут быть неоправданны. Однако если продукт планируется поддерживать и развивать годами, с возможным расширением команды разработчиков, то правильная архитектура с самого начала экономит значительные ресурсы в долгосрочной перспективе.

Третий аспект — количество и разнородность источников данных. Проект, работающий с единственным REST API, архитектурно проще, чем приложение, которое одновременно интегрируется с несколькими внешними сервисами, использует локальную базу данных для оффлайн-режима и кэширует данные в памяти для производительности. Каждый дополнительный источник данных увеличивает сложность координации, синхронизации и обработки ошибок.

Четвертый критерий — требования к тестируемости и возможности командной работы. Если над проектом работает несколько разработчиков, архитектура должна позволять разбивать систему на относительно независимые модули, над которыми можно работать параллельно. Если требуется высокое покрытие кода автоматическими тестами (unit, integration), архитектура должна обеспечивать возможность изоляции компонентов для тестирования.

Для визуализации этих критериев можно использовать следующую таблицу принятия решений:

На практике многие учебные и коммерческие проекты попадают в категорию средней сложности, что делает Clean Architecture оптимальным выбором — она обеспечивает достаточную структурированность без чрезмерного усложнения.

2. Базовое распределение архитектуры на слои

После определения необходимого уровня архитектурной сложности следующим шагом становится понимание и реализация базового принципа разделения ответственности (Separation of Concerns). На этом этапе мы рассматриваем архитектуру на самом общем уровне, разбивая приложение на три фундаментальных слоя: UI, Domain и Data. Это основа, на которой впоследствии будут строиться более сложные паттерны, такие как Clean Architecture и Repository, но пока мы фокусируемся на понимании роли каждого слоя и направлении зависимостей между ними.

UI Layer (Слой пользовательского интерфейса) отвечает за всё, что видит и с чем взаимодействует конечный пользователь. Это не только визуальные компоненты (виджеты, экраны, анимации), но и логика, непосредственно связанная с откликом интерфейса: обработка нажатий, валидация ввода в полях, управление состоянием загрузки или ошибки на экране. Ключевая задача этого слоя — быть "глупым" исполнителем: он получает от Domain Layer готовые к отображению данные и команды (например, "показать список задач" или "отобразить ошибку 'Неверный пароль'") и преобразует действия пользователя (тап по кнопке "Сохранить") в четкие запросы к бизнес-логике. На данном этапе важно зафиксировать, что UI Layer не должен содержать правил о том, как рассчитывается скидка, откуда берутся данные или как валидируется email — это ответственность других слоев.

Domain Layer (Слой предметной области, или бизнес-логики) — это концептуальное ядро приложения. Он содержит правила, сущности и процессы, которые определяют, как работает ваш бизнес или предметная область, независимо от того, является ли приложение мобильным, веб- или десктопным. Здесь живут Models (часто называемые бизнес-моделями или сущностями) — классы, представляющие ключевые понятия (Пользователь, Заказ, Товар) вместе с их инвариантами и базовыми методами (например, `user.isAdult()`). Domain Layer максимально изолирован: он ничего не знает о Flutter, базах данных, API или любых других внешних технологиях. Его можно собрать и протестировать как обычную Dart-библиотеку. Зависимости направлены внутрь: Domain Layer не зависит от UI и Data слоев.

Data Layer (Слой данных) — это шлюз приложения к внешнему миру. Его единственная задача — предоставлять Domain Layer необходимые данные в том формате, в котором он их ожидает

(в виде тех самых Models). На текущем, базовом уровне понимания архитектуры, мы рассматриваем Data Layer как монолитный компонент, который знает, как получить данные (например, сделать HTTP-запрос к конкретному API или прочитать файл) и как преобразовать сырой ответ (JSON, строка базы данных) в бизнес-модель. Позже, при изучении Clean Architecture и паттерна Repository, мы разделим этот слой на data_sources и dto (Data Transfer Objects), чтобы абстрагироваться от конкретных источников, но пока мы говорим об общей ответственности.

Наглядное представление потоков данных и зависимостей между слоями на этом этапе показано на рисунке 1.

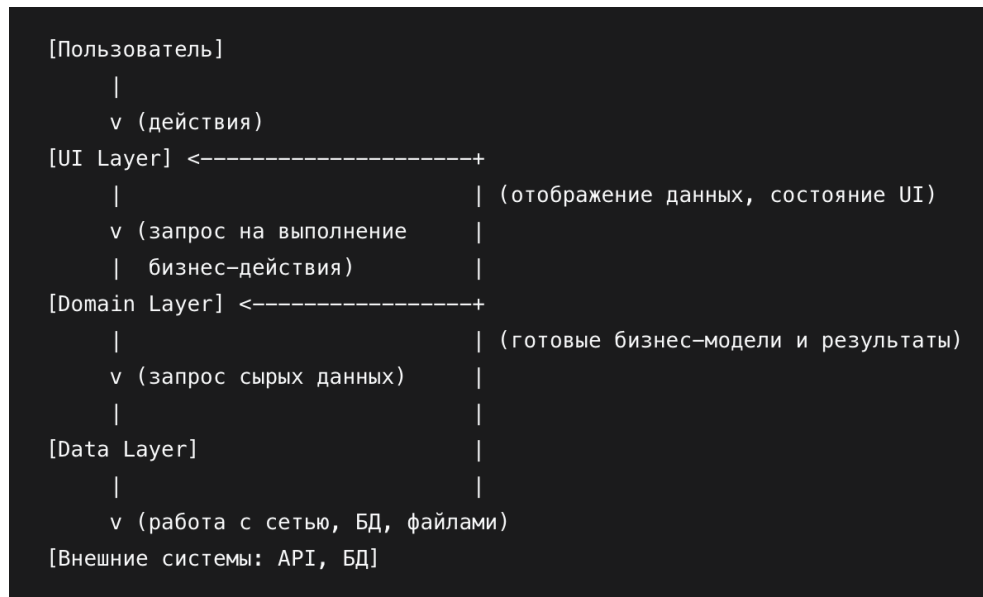


Рисунок 1 – взаимодействие разных слоев базовой структуры

Направление зависимостей строго однонаправлено: UI зависит от Domain, Domain зависит от абстракций Data (пока — условно), Data зависит от внешних систем. Это гарантирует, что изменения в способе отображения (например, переход от ListTile к Card) или в источнике данных (замена REST API на GraphQL) не потребуют переписывания бизнес-правил.

Пример простейшей структуры проекта, отражающей это базовое разделение можно увидеть на рисунке 2.

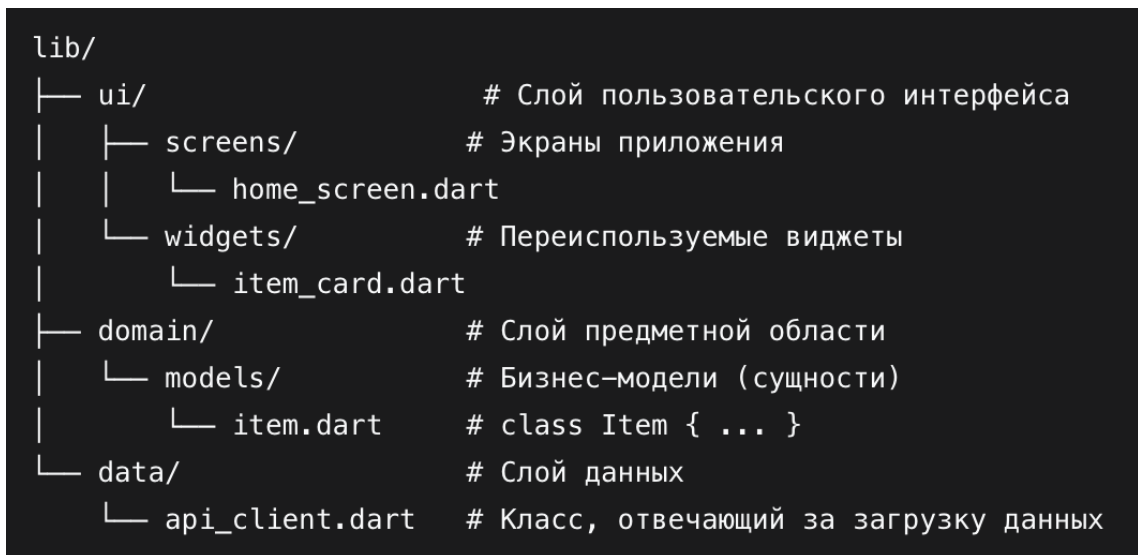


Рисунок 2 – Простейшая структура приложения, построенного на основании базовых принципов архитектурирования

В этой структуре `home_screen.dart` (UI) импортирует `item.dart` (Domain) для отображения данных и использует `api_client.dart` (Data) для их получения через вызов метода в `initState()` или по нажатию кнопки. Модель `Item` при этом остается "чистой" и не знает о своем происхождении.

Таким образом, базовое распределение на три слоя устанавливает фундаментальные границы в приложении.

3. Применение архитектуры Clean Architecture

Архитектура Clean Architecture (Чистая Архитектура), предложенная Робертом Мартином, представляет собой эволюцию базовой трёхслойной модели, доводя принцип разделения ответственности и инверсии зависимостей до логического завершения. Её основная цель — создание системы, в которой бизнес-правила и сущности (ядро приложения) полностью изолированы от деталей реализации: фреймворков, баз данных, пользовательских интерфейсов и внешних сервисов. Это достигается путём организации кода в виде концентрических кругов, где каждый внутренний круг содержит более абстрактные и стабильные политики, а внешние — конкретные детали. Зависимости направлены строго внутрь: код внешних кругов может зависеть от внутренних, но не наоборот.

В адаптации для Flutter-приложения это приводит к уточнению и реструктуризации базовых слоёв (UI, Domain, Data) в более строгую систему. Domain Layer становится абсолютно чистым и независимым, обрастающим чётко определёнными интерфейсами для коммуникации с внешним миром. Data и UI слои превращаются в "плагины" к этому ядру, реализующие эти интерфейсы.

3.1 Внутренний круг: Domain Layer (Модели и Сценарии Использования)

Это ядро системы, содержащее бизнес-логику. Оно делится на два ключевых компонента:

- **Models** (Бизнес-модели) — это сущности предметной области, которые инкапсулируют самые общие и высокоуровневые правила. Они представляют собой простые классы Dart без каких-либо внешних зависимостей. Важно подчеркнуть, что на этом уровне мы говорим именно о бизнес-моделях (Models), а не о "сущностях"

базы данных (Entities)", так как их цель — отражать концепции предметной области (например, "Счёт", "Пользователь", "Транзакция"), а не структуру таблиц в БД. Пример такой структуры можно изучить на рисунке 3.

```
// lib/domain/models/account.dart
// Бизнес-модель "Счёт", не зависящая от фреймворков и баз данных
class Account {
  final String id;
  final String ownerName;
  final double balance;
  final DateTime createdAt;

  Account({
    required this.id,
    required this.ownerName,
    required this.balance,
    required this.createdAt,
  });

  // Бизнес-правило, инкапсулированное в модели
  bool canWithdraw(double amount) {
    return balance >= amount && amount > 0;
  }

  // Бизнес-операция, возвращающая новую модель
  Account withDebit(double amount) {
    if (!canWithdraw(amount)) {
      throw InsufficientFundsException();
    }
    return Account(
      id: id,
      ownerName: ownerName,
      balance: balance - amount,
      createdAt: createdAt,
    );
  }
}
```

Рисунок 3 – Бизнес модель счета с набором простейших модифицирующих методов

- Use Cases (Сценарии использования). Каждый Use Case инкапсулирует одну конкретную бизнес-операцию (например, "Перевести деньги", "Получить выписку по счёту"). Use Case координирует поток данных между Models и интерфейсами внешнего слоя (о которых будет сказано ниже), применяя бизнес-правила. Он зависит только от Models и абстрактных интерфейсов для получения данных. Пример такого класса можно изучить на рисунке 4.

```
// lib/domain/usecases/transfer_money_usecase.dart
import '../models/account.dart';

// Абстрактный интерфейс, который Use Case ожидает "снаружи"
// Пока это просто контракт, реализация будет во внешнем круге.
abstract class AccountDataSource {
  Future<Account> getAccount(String id);
  Future<void> updateAccount(Account account);
}

// Сам Use Case
class TransferMoneyUseCase {
  final AccountDataSource dataSource; // Зависимость от абстракции

  TransferMoneyUseCase(this.dataSource);

  Future<void> execute({
    required String fromAccountId,
    required String toAccountId,
    required double amount,
  }) async {
    // 1. Получаем модели через абстракцию
    final fromAccount = await dataSource.getAccount(fromAccountId);
    final toAccount = await dataSource.getAccount(toAccountId);

    // 2. Применяем бизнес-правила (используем методы модели)
    if (!fromAccount.canWithdraw(amount)) {
      throw TransferException('Insufficient funds');
    }

    // 3. Создаём новые состояния моделей
    final updatedFromAccount = fromAccount.withDebit(amount);
    final updatedToAccount = toAccount.copyWith(
      balance: toAccount.balance + amount,
    );

    // 4. Сохраняем изменения через абстракцию
    await dataSource.updateAccount(updatedFromAccount);
    await dataSource.updateAccount(updatedToAccount);
  }
}
```

Рисунок 4 – Протокол и реализация класса бизнес логики работы с банковским аккаунтом

3.2 Внешние круги: Data Layer и UI Layer (Реализации и детали)

Внешние круги содержат конкретные реализации абстракций, ожидаемых ядром, а также все детали фреймворков.

- Data Layer (Слой данных): На этом этапе мы превращаем его из монолитного компонента в слой, который реализует интерфейсы данных (Data Source Interfaces), определённые в Domain Layer. Пока мы не вводим паттерн Repository, этот слой может быть представлен конкретными классами, которые напрямую реализуют эти интерфейсы, работая, например, с API или локальной БД. Важно, что эти классы знают все технические детали (Dio, SQLite, JSON) и содержат DTO (Data Transfer

Objects) для преобразования сетевых/базовых форматов в бизнес-модели. Пример реализации DTO структуры и класса Data Layer слоя можно рассмотреть на рисунке 5 и 6 соответственно.

```
// lib/data/datasources/account_api_datasource.dart
import 'package:dio/dio.dart';
import '../..domain/models/account.dart';
import '../..domain/usecases/transfer_money_usecase.dart'; // Импорт интерфейса!

// DTO для парсинга ответа API
class ApiAccountDTO {
  final String account_id;
  final String owner;
  final double current_balance;
  final String created_at;

  ApiAccountDTO({...});
  Account toModel() { // Преобразование DTO в бизнес-модель
    return Account(
      id: account_id,
      ownerName: owner,
      balance: current_balance,
      createdAt: DateTime.parse(created_at),
    );
  }
}
```

Рисунок 5 – пример DTO структуры

```
// Конкретная реализация интерфейса из Domain Layer
class AccountApiDataSource implements AccountDataSource {
  final Dio dio;

  @override
  Future<Account> getAccount(String id) async {
    final response = await dio.get('/accounts/$id');
    final dto = ApiAccountDTO.fromJson(response.data);
    return dto.toModel(); // Возвращаем Domain Model
  }

  @override
  Future<void> updateAccount(Account account) async {
    // Преобразование модели в DTO для отправки
    final dto = ApiAccountDTO.fromModel(account);
    await dio.put('/accounts/${account.id}', data: dto.toJson());
  }
}
```

Рисунок 6 – пример класса Data layer слоя, описывающий логику работы с данными из источника

- UI Layer (Слой представления): Этот слой также становится "плагином" к ядру. Он зависит от Use Cases и Models. Его ответственность — инициировать выполнение

Use Cases (например, по нажатию кнопки) и отображать полученные Models. Вся специфика Flutter (Widget, BuildContext, State Management) содержится здесь.

Обновлённая структура проекта с учётом применения Clean Architecture изображена на рисунке 7.



Рисунок 7 – архитектурная схема приложения, построенная на паттерне чистой архитектуры

Ключевое отличие от базовой трёхслойной архитектуры теперь очевидно:

- Domain Layer совершенно чист. Он не содержит даже намёка на Flutter или Dio. Вместо этого он определяет интерфейсы (interfaces), через которые будет получать данные (например, AccountDataSource).
- Data Layer не просто предоставляет данные, а является конкретной реализацией (AccountApiDataSource) этих доменных интерфейсов.
- Направление зависимостей формализовано: AccountApiDataSource (Data) → реализует → AccountDataSource (Domain Interface). TransferMoneyUseCase (Domain) зависит только от → AccountDataSource (абстракция). TransferScreen (UI) зависит от → TransferMoneyUseCase (Domain).

Таким образом, Clean Architecture закрепляет статус Domain Layer как независимого, тестируемого и стабильного ядра приложения. На следующем этапе, при изучении паттерна

Repository, мы усовершенствуем Data Layer, добавив ещё один уровень абстракции для более гибкого управления источниками данных.

4. Реализация слоя Models

Реализация слоя моделей представляет собой следующий этап архитектурного совершенствования, где мы фокусируемся на максимальной инкапсуляции и управлении зависимостями. Если в рамках Clean Architecture Models были частью Domain Layer, то теперь мы выделяем их в абсолютно независимое, изолированное пространство. Это решение обусловлено критически важной ролью бизнес-моделей: они являются центральным связующим звеном, на которое опираются все остальные слои приложения. Use Cases строят свою логику вокруг них, UI Layer отображает их состояние, а Data Layer обязан уметь преобразовывать сырые данные из внешних источников именно в эти модели и обратно.

Выделение Models в отдельный слой преследует несколько стратегических целей:

- Устранение циклических зависимостей: В сложных проектах Domain Layer (с его Use Cases) и Data Layer могут требовать взаимных ссылок. Use Case зависит от Model, а Data Source (например, маппер) также зависит от Model для преобразования. Расположение Model внутри Domain создает потенциальную петлю. Вынос Models в отдельный слой разрывает эту возможную связь, делая Models общим, нейтральным фундаментом.
- Чистота управления зависимостями (Dependency Management): В экосистеме Dart/Flutter управление зависимостями через pubspec.yaml является ключевым. Выделение Models в отдельный пакет (или, как минимум, в изолированный модуль внутри монорепоzitория) позволяет чётко контролировать, кто может на них ссылаться. Это предотвращает ситуацию, когда код UI или Data слоя по ошибке начинает импортировать и использовать Use Cases или элементы бизнес-логики напрямую, нарушая поток зависимостей.
- Повышение переиспользуемости: Бизнес-модели, представляющие основные концепции предметной области (Пользователь, Заказ, Товар), зачастую стабильны и могут использоваться в нескольких независимых модулях или даже в разных приложениях внутри одной продуктовой экосистемы (например, в мобильном приложении и бэкенд-сервисе на Dart). Их изоляция упрощает такую миграцию и совместное использование.
- Инкапсуляция источников данных: DTO и их преобразования (Mappers) должны быть полностью скрыты внутри конкретного DataSource. Это гарантирует, что специфика работы с конкретным API или базой данных не просачивается в другие части Data Layer. Каждый источник данных полностью отвечает за получение сырых данных, их парсинг в DTO и преобразование в бизнес-модели с помощью своих собственных Mappers.

Структура проекта с выделенным слоем Models и инкапсулированными Mappers можно увидеть на рисунке 8.

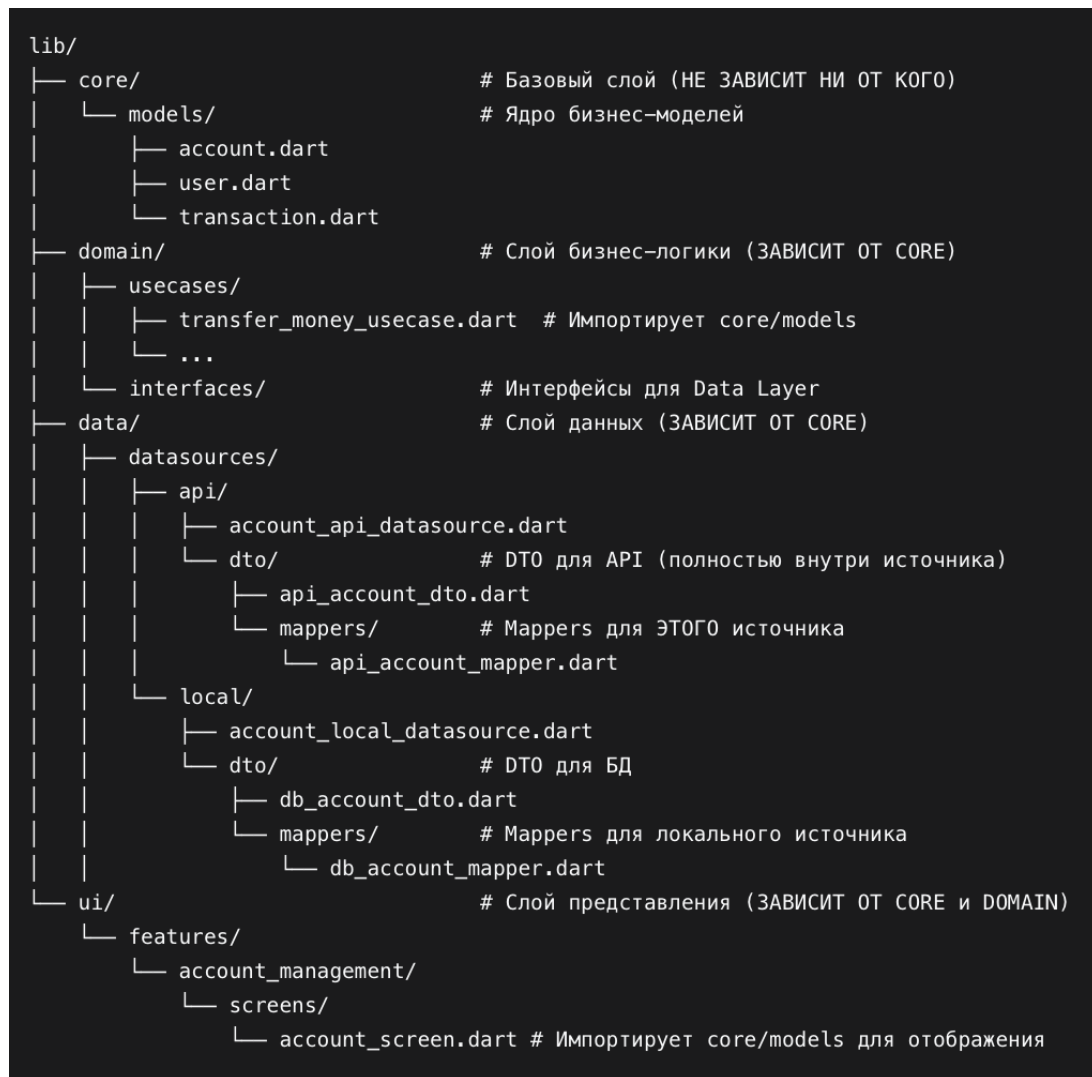


Рисунок 8 – структура приложения при вынесении слоя Models

На схеме проекта можно увидеть новые классы, относящиеся к подслою mappers. Их ключевая идея заключается в том, что каждый DataSource полностью отвечает за свою "зону ответственности". DTO и соответствующие Mappers являются приватной реализационной деталью конкретного источника данных и не должны использоваться за его пределами. Это достигается путем расположения DTO и Mappers внутри подпапки конкретного DataSource.

Рассмотрим пример некой бизнес-сущности, DTO и Mapper для API источника на рисунках 9, 10, 11 и 12 соответственно.

```
// lib/core/models/account.dart
class Account {
  final String id;
  final String ownerName;
  final double balance;
  final DateTime createdAt;

  Account({
    required this.id,
    required this.ownerName,
    required this.balance,
    required this.createdAt,
  });

  bool get isOverdrawn => balance < 0;

  Account copyWith({
    String? id,
    String? ownerName,
    double? balance,
    DateTime? createdAt,
  }) {
    return Account(
      id: id ?? this.id,
      ownerName: ownerName ?? this.ownerName,
      balance: balance ?? this.balance,
      createdAt: createdAt ?? this.createdAt,
    );
  }
}
```

Рисунок 9 – бизнес-сущность, используемая в приложении

```
// lib/data/datasources/api/dto/api_account_dto.dart
// DTO полностью отражает структуру ответа API
class ApiAccountDTO {
  final String account_id; // snake_case из API
  final String owner_name;
  final double current_balance;
  final String created_at_iso;

  ApiAccountDTO({
    required this.account_id,
    required this.owner_name,
    required this.current_balance,
    required this.created_at_iso,
  });

  factory ApiAccountDTO.fromJson(Map<String, dynamic> json) {
    return ApiAccountDTO(
      account_id: json['account_id'] as String,
      owner_name: json['owner_name'] as String,
      current_balance: (json['current_balance'] as num).toDouble(),
      created_at_iso: json['created_at_iso'] as String,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'account_id': account_id,
      'owner_name': owner_name,
      'current_balance': current_balance,
      'created_at_iso': created_at_iso,
    };
  }
}
```

Рисунок 10 – DTO структура, описывающая модель данных в рамках некого Data Source.

```
// lib/data/datasources/api/dto/mappers/api_account_mapper.dart
import '../../../../../core/models/account.dart'; // Импорт модели из CORE
import '../api_account_dto.dart';

// Mapper как расширение для DTO, инкапсулированное внутри источника API
extension ApiAccountMapper on ApiAccountDTO {
  // Преобразование DTO API в бизнес-модель
  Account toModel() {
    return Account(
      id: account_id,
      ownerName: owner_name,
      balance: current_balance,
      createdAt: DateTime.parse(created_at_iso),
    );
  }
}

// Обратное преобразование может быть как расширением для модели,
// но остаётся в пространстве имён этого источника данных
extension AccountToApiMapper on Account {
  ApiAccountDTO toApiDto() {
    return ApiAccountDTO(
      account_id: id,
      owner_name: ownerName,
      current_balance: balance,
      created_at_iso: createdAt.toIso8601String(),
    );
  }
}
}
```

Рисунок 11 – пример класса расширения марпер, преобразующего DTO структуру в бизнес-сущность для передачи далее в систему

```
// lib/data/datasources/api/account_api_datasource.dart
import 'package:dio/dio.dart';
import '../../../../../core/models/account.dart'; // Импорт модели
import 'dto/api_account_dto.dart';
import 'dto/mappers/api_account_mapper.dart'; // Импорт ЧАСТНОГО марпера

class AccountApiDataSource {
  final Dio dio;

  Future<Account> getAccount(String id) async {
    final response = await dio.get('/api/v1/accounts/$id');

    // 1. Парсим JSON в DTO (инкапсулированная логика API)
    final dto = ApiAccountDTO.fromJson(response.data);

    // 2. Используем инкапсулированный марпер для преобразования
    return dto.toModel(); // Метод из ApiAccountMapper
  }

  Future<void> updateAccount(Account account) async {
    // 1. Используем инкапсулированный марпер для обратного преобразования
    final requestDto = account.toApiDto(); // Метод из AccountToApiMapper

    // 2. Отправляем DTO
    await dio.put('/api/v1/accounts/${account.id}', data: requestDto.toJson());
  }
}
}
```

Рисунок 12 – Применение инкапсулированного Mapper в DataSource

В чем же заключаются преимущества такого подхода:

- Полная инкапсуляция: Специфика работы с API (названия полей, форматы дат) остаётся внутри `datasources/api/`. Если API изменится, правки нужно будет делать только в этой директории.
- Отсутствие утечек абстракций: Другие части приложения (например, Domain Layer или другие DataSource) не знают о существовании `ApiAccountDTO` или `ApiAccountMapper`. Они работают только с бизнес-моделями из `core/models/`.
- Гибкость для разных источников: Каждый DataSource может иметь свои собственные DTO и Mappers, оптимизированные под конкретный протокол или технологию. Например, локальный источник данных для SQLite может использовать совершенно другие DTO и Mappers, не связанные с API.
- Упрощение тестирования: DataSource можно тестировать изолированно, подменяя только DTO и Mappers, без необходимости создавать сложные JSON-структуры на уровне бизнес-моделей.

Таким образом, вынос Models в отдельный слой `core/models` и строгая инкапсуляция DTO с Mappers внутри соответствующих DataSource создаёт архитектуру с максимально чёткими границами ответственности. Это обеспечивает высокую степень поддерживаемости, тестируемости и гибкости приложения при изменении внешних интерфейсов или добавлении новых источников данных.

5. Применение архитектуры Repository

Архитектура Repository представляет собой завершающий этап в построении отказоустойчивой и гибкой системы управления данными. На предыдущих этапах мы создали чистые бизнес-модели (`core/models`), выделили сценарии использования (`domain/usecases`) и инкапсулировали работу с конкретными источниками данных в отдельных DataSource (`data/datasources/`). Однако оставалась проблема: Use Cases всё ещё зависели от конкретных интерфейсов DataSource, что ограничивало возможность одновременной работы с несколькими источниками данных и усложняло реализацию сложных стратегий доступа к данным (например, "сначала кэш, потом сеть"). Чтобы понять суть проблемы, давайте проследим эволюцию подхода к работе с данными:

- Наивный подход: Use Case напрямую вызывал API-клиент, смешивая бизнес-логику с деталями сетевых запросов
- Clean Architecture: Use Case стал зависеть от абстрактного DataSource, но каждый Use Case знал о конкретных источниках данных
- Repository Pattern: Use Case зависит от единой абстракции Repository, которая инкапсулирует ВСЕ источники данных и стратегии работы с ними

Паттерн Repository решает эту проблему, вводя дополнительный уровень абстракции между Domain Layer (бизнес-логикой) и Data Layer (конкретными источниками данных). Repository выступает в роли единого фасада для доступа к данным, скрывая от бизнес-логики все детали о том,

откуда и как данные поступают: из сети, локальной базы данных, памяти или их комбинации. Главная проблема, которую решает Repository — это координация нескольких источников данных без усложнения бизнес-логики. В реальном приложении данные редко приходят из одного места. Типичный сценарий включает:

- Основной сетевой источник (например, REST API) для актуальных данных
- Локальное хранилище для работы в оффлайн-режиме или кэширования
- Промежуточный кэш в памяти для быстрого повторного доступа

Без Repository каждый Use Case должен был бы самостоятельно управлять этой сложностью, что приводило бы к дублированию кода и нарушению принципа единой ответственности.

Ключевое отличие от предыдущего подхода: Use Case теперь ничего не знает о существовании разных источников данных. Он просто говорит: "Мне нужен счёт с ID 123", а Repository решает, откуда его взять и как объединить данные, если это необходимо. Принцип схемы работы системы при внедрении паттерна Repository можно увидеть на рисунке 13.

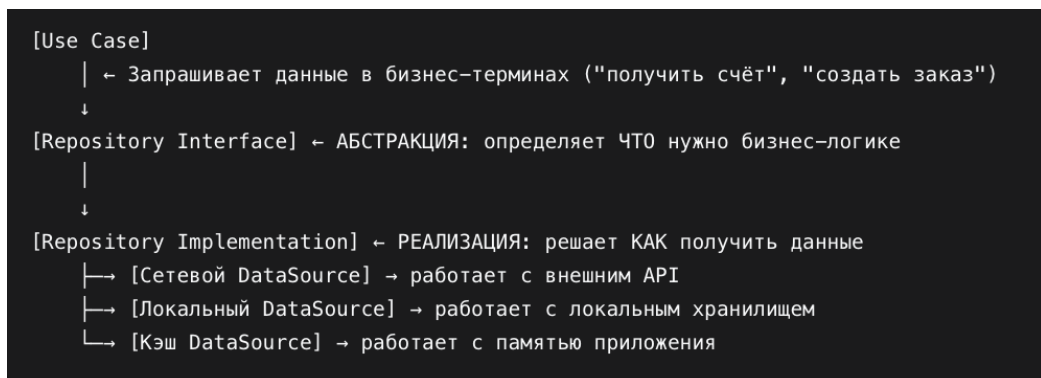


Рисунок 13 - Концептуальная схема работы Repository

При внедрении паттерна Repository в архитектуру приложения получается схема, проиллюстрированная на рисунке 14.

```

lib/
├── core/
│   └── models/
│       └── account.dart # Бизнес-модель Account
├── domain/
│   ├── interfaces/
│   │   └── repositories/
│   │       └── account_repository.dart # Абстрактный интерфейс
│   └── usecases/
│       └── get_account_usecase.dart # Использует AccountRepository
├── data/
│   ├── repositories/
│   │   └── account_repository_impl.dart # Реализация Repository
│   └── datasources/
│       ├── api/
│       │   ├── account_api_datasource.dart
│       │   └── dto/
│       │       ├── api_account_dto.dart
│       │       └── mappers/
│       │           └── api_account_mapper.dart
│       └── local/
│           ├── account_local_datasource.dart
│           └── dto/
│               ├── db_account_dto.dart
│               └── mappers/
│                   └── db_account_mapper.dart
└── ui/
    ├── features/
    │   └── account_management/
    │       ├── screens/
    │       │   └── account_screen.dart
    │       └── delegates/
    │           └── account_delegate.dart

```

Рисунок 14 – схема приложения при внедрении паттерна Repository

5.1 Абстрактный интерфейс Repository: язык бизнеса, а не технологий

В Domain Layer мы создаём интерфейс, который говорит исключительно на языке предметной области. Это критически важно, потому что этот интерфейс определяет контракт между бизнес-логикой и механизмами доступа к данным без привязки к каким-либо технологиям. Пример такого абстрактного класса можно увидеть на рисунке 15.

```
// lib/domain/interfaces/repositories/account_repository.dart
import '...../core/models/account.dart';

// Этот интерфейс определяет КОНТРАКТ между бизнес-логикой и механизмами доступа к данным
// Обратите внимание: нет упоминаний о конкретных технологиях (API, БД, кэш)
abstract class AccountRepository {
  // Все методы оперируют бизнес-концепциями и бизнес-моделями

  // Получение счёта по идентификатору
  Future<Account> getAccountById(String id);

  // Получение всех счетов пользователя
  Future<List<Account>> getUserAccounts(String userId);

  // Создание нового счёта
  Future<Account> createAccount(Account account);

  // Обновление существующего счёта
  Future<Account> updateAccount(Account account);

  // Удаление счёта
  Future<void> deleteAccount(String id);

  // Специфичные бизнес-методы, которые могут требовать
  // агрегации данных или сложной логики доступа
  Future<List<Account>> getOverdueAccounts();
  Future<double> getTotalBalance(String userId);
}
```

Рисунок 15 – пример абстрактного контракта repository класса

5.2 Реализация Repository: координация источников данных

Реализация Repository находится в Data Layer и содержит всю логику работы с конкретными источниками данных. С примером реализации класса можно ознакомиться на рисунках 16, 17 и 18.

```
// lib/data/repositories/account_repository_impl.dart
import '...../domain/interfaces/repositories/account_repository.dart';
import '...../core/models/account.dart';
import '...../datasources/api/account_api_datasource.dart';
import '...../datasources/local/account_local_datasource.dart';

class AccountRepositoryImpl implements AccountRepository {
  final AccountApiDataSource _apiDataSource;
  final AccountLocalDataSource _localDataSource;

  AccountRepositoryImpl({
    required AccountApiDataSource apiDataSource,
    required AccountLocalDataSource localDataSource,
  }) : _apiDataSource = apiDataSource,
       _localDataSource = localDataSource;

  @override
  Future<Account> getAccountById(String id) async {
    // Реализация стратегии "сначала сеть, потом локальное хранилище"
    try {
      // Пытаемся получить данные из сетевого источника
      final account = await _apiDataSource.getAccount(id);

      // Сохраняем полученные данные в локальное хранилище
      await _localDataSource.saveAccount(account);

      return account;
    } catch (networkError) {
      // Если сеть недоступна, используем локальное хранилище
      return await _localDataSource.getAccount(id);
    }
  }
}
```

Рисунок 16 – Класс реализации протокола Repository. Часть 1.


```

@Override
Future<List<Account>> getUserAccounts(String userId) async {
    // Реализация стратегии "быстрый локальный ответ + фоновая синхронизация"

    // Сразу возвращаем локальные данные для быстрого отклика UI
    final localAccounts = await _localDataSource.getUserAccounts(userId);

    // В фоне обновляем данные из сети
    Future.microtask(() async {
        try {
            final remoteAccounts = await _apiDataSource.getUserAccounts(userId);
            await _localDataSource.saveUserAccounts(remoteAccounts);
        } catch (e) {
            // Обработка ошибок фоновой синхронизации
        }
    });

    return localAccounts;
}

@Override
Future<double> getTotalBalance(String userId) async {
    // Пример агрегации данных: Repository может выполнять сложные операции
    final accounts = await getUserAccounts(userId);

    // Бизнес-расчёт: сумма балансов всех счетов
    return accounts.fold(
        0.0,
        (sum, account) => sum + account.balance
    );
}

```

Рисунок 16 – Класс реализации протокола Repository. Часть 2.

```

@Override
Future<Account> createAccount(Account account) async {
    // Пример стратегии "оптимистичное создание"
    try {
        // Сначала сохраняем локально
        await _localDataSource.saveAccount(account);

        // Пытаемся создать на сервере
        final createdAccount = await _apiDataSource.createAccount(account);

        // Обновляем локальную версию данными с сервера
        await _localDataSource.saveAccount(createdAccount);

        return createdAccount;
    } catch (e) {
        // Обработка ошибок при создании
        rethrow;
    }
}

// Реализация остальных методов интерфейса...
}

```

Рисунок 16 – Класс реализации протокола Repository. Часть 3.

5.3 Использование Repository в Use Case

Use Case теперь работает только с абстрактным Repository, не зная о деталях получения данных. Способ применения можно изучить на рисунке 17.

```

// lib/domain/usecases/get_account_usecase.dart
import '../core/models/account.dart';
import '../interfaces/repositories/account_repository.dart';

class GetAccountUseCase {
  final AccountRepository _repository;

  GetAccountUseCase(this._repository);

  Future<Account> execute(String accountId) async {
    // Валидация входных данных (бизнес-правило)
    if (accountId.isEmpty) {
      throw ArgumentError('Идентификатор счёта не может быть пустым');
    }

    // Получение данных через Repository
    final account = await _repository.getAccountById(accountId);

    // Применение бизнес-правил
    if (account.isClosed) {
      throw Exception('Счёт закрыт и недоступен для операций');
    }

    return account;
  }
}

```

Рисунок 17 – применение методов Repository в классе Use Case

Задание на практическую работу

В качестве задания на практическую работу, студенту в индивидуальном порядке требуется подготовить следующее задание.

Студенту необходимо переработать имеющееся приложение, выполненное на предыдущих практических работах, таким образом, чтобы в нем была реализована корректная архитектура, подходящая ему по критериям сложности.

Студенту требуется выполнить все пункты в плане практической работы с конспектированием своих действий, а также фотофиксацией контрольных точек в отчете о практической работе.